

Trabajo Tutelado AII:
Infraestructura declarativa reproducible
usando OpenTofu y NixOS

Autores:

Mario Lamas Angeriz (m.lamasa@udc.es)

Alexandre Borrazás Mancebo (alexandre.bmancebo@udc.es)

[Repositorio en GitHub](#)

Índice

1. Propuesta	2
2. Objetivos	2
3. Hardware empleado	2
4. Tecnologías usadas	2
5. Diagrama de red y estructura base	3
6. Estructura del proyecto	4
7. Desarrollo del proyecto	6
7.1. Configuraciones iniciales	6
7.2. Declaración de los recursos	6
7.3. Creación de la plantilla base	7
7.4. Instalación de NixOS con nixos-anywhere	8
7.5. Gestión de cambios con Colmena	9
7.5.1. Integración con el flake	9
7.5.2. Flujo de trabajo habitual	10
7.5.3. Ventajas operativas	10
7.5.4. Configuración SSH para Colmena	10
7.6. Configuración de NixOS	11
7.6.1. Estructura del flake	11
7.6.2. Módulos compartidos	11
7.6.3. Configuración de hosts	12
8. Servicios desplegados	12
8.1. Caddy (proxy inverso)	12
8.2. DDNS (ddns.nix)	12
8.3. Workers con Nginx (nginx_worker.nix)	12
8.4. Docker (docker.nix)	12
8.5. Monitorización: Prometheus, Node Exporter y Grafana	13
9. Ventajas y limitaciones	13
9.1. Ventajas	13
9.2. Limitaciones y desafíos	13
10. Conclusiones	14
10.1. Pipeline de despliegue	14
10.2. Valoración global	14

1. Propuesta

Aprender más sobre **Infrastructure as Code (IaC)** usando herramientas como **Terraform** [1] o **OpenTofu** [2] que permiten describir y desplegar infraestructura de forma declarativa, mientras que **NixOS** [3] proporciona un sistema operativo completamente reproducible cuya configuración también se define mediante código.

Este trabajo propone explorar la integración de ambas tecnologías para crear una infraestructura completamente declarativa, donde tanto la creación de recursos como la configuración del sistema operativo estén definidas en repositorios versionados ([GitHub](#)).

2. Objetivos

El objetivo principal del trabajo es diseñar e implementar una plataforma de infraestructura reproducible basada en principios de **Infrastructure as Code**.

- Estudiar el paradigma de infraestructura declarativa.
- Analizar el funcionamiento de **Terraform** [1] y **OpenTofu** [2] para la provisión de recursos.
- Analizar el modelo de configuración declarativa de **NixOS** [3].
- Diseñar una arquitectura donde **Terraform** [1] o **OpenTofu** [2] gestione la infraestructura y **NixOS** [3] configure los sistemas.
- Implementar un entorno reproducible donde la infraestructura pueda desplegarse automáticamente a partir del código.
- Evaluar las ventajas y limitaciones del enfoque propuesto.

3. Hardware empleado

Para la realización del proyecto contamos con un servidor rack **Dell** cedido por un amigo, cuyas especificaciones técnicas relevantes para el proyecto son las siguientes:

- **CPU:** Intel[®] Xeon[®] E5-2420 v2 @ 2.20GHz — 12 cores (1 socket)
- **RAM:** 32 GiB
- **Almacenamiento:** 6 discos duros extraíbles *hot-swappable* de 136 GB cada uno, configurados en **RAID 1+0**.

4. Tecnologías usadas

Para la realización del proyecto decidimos emplear las siguientes tecnologías:

- **Proxmox VE** [4]: plataforma de virtualización open-source que utilizamos como hipervisor para la gestión de nuestras máquinas virtuales.
- **pfSense** [5]: firewall y router virtualizado que empleamos para la gestión de las redes del proyecto.
- **Tailscale** [6]: VPN de configuración cero utilizada como acceso remoto de respaldo.

- **OpenTofu** [2]: fork de Terraform open-source (sus ficheros de configuración son compatibles), herramienta de IaC que empleamos para la declaración y asignación de recursos de nuestras máquinas virtuales.
- **NixOS** [3]: distribución de Linux que emplea el gestor de paquetes Nix, gracias a su configuración declarativa nos permite desplegar en las máquinas virtuales todo tipo de servicios de forma sencilla y automatizada.
- **nixos-anywhere** [7]: herramienta de instalación de NixOS, nos permite realizar la instalación del SO de forma remota mediante `ssh` y de forma completamente automatizada.
- **Colmena** [8]: tras la primera build de los sistemas de NixOS, Colmena nos permite realizar cambios sobre el SO (mientras este se ejecuta) de forma remota y sencilla sin necesidad de reinstalar.
- **agenix** [9]: paquete que permite la gestión de secretos mediante cifrado `age` [10], nos permite crear secretos y subirlos a un repositorio sin miedo a exponer información sensible.
- **Docker** [11]: mediante la configuración en NixOS, podemos desplegar servicios sobre Docker en boot lanzándolos de forma automatizada.
- **Caddy** [12]: servidor web y proxy inverso con gestión automática de certificados TLS.
- **Nginx** [13]: servidor HTTP utilizado en los workers para servir páginas de índice.
- **Prometheus** [14]: sistema de monitorización y base de datos de series temporales.
- **Grafana** [15]: plataforma de observabilidad para visualización de métricas.
- **WireGuard** [16]: protocolo VPN moderno y seguro para el acceso remoto a la red.
- **Disko** [17]: herramienta de particionado declarativo de discos para NixOS.
- **fail2ban** [18]: software de prevención de intrusiones que protege los servicios SSH.

5. Diagrama de red y estructura base

En primer lugar, instalamos Proxmox VE [4] como hipervisor en el servidor para poder virtualizar las máquinas necesarias para el proyecto.

A continuación, decidimos definir la estructura de red, y equipos/contenedores a crear.

Antes de empezar a desarrollar el código de IaC, era necesario tener una red funcional que nos permitiese trabajar con las máquinas virtuales de forma cómoda y segura.

Para ello reemplazamos el router de nuestro piso por un router virtualizado con **pfSense** [5] y creamos 2 contenedores LXC [19] Debian para las VPN, uno con **Tailscale** [6] y otro con **WireGuard** [16]. El **Tailscale** está de *fallback*, es decir, por si el WireGuard falla, siempre tenemos una forma de acceder a la red de forma remota.

Ahora en cuanto a la red, decidimos crear 3 redes, como se puede ver en la [Figura 1](#), se creó una red **DMZ** para poder aislar las máquinas y reproducir de forma más fiable una red empresarial de producción. La red **LAN** es la red habitual que todos tenemos en casa, aquí simulamos la intranet de una organización y la **WAN** representa internet.

Como solo disponemos de 1 dirección IP en el router hemos redirigido el puerto **3478/udp** hacia nuestro WireGuard y además los puertos **80/tcp/443/tcp/443/udp** hacia nuestro *reverse proxy*, en este caso **Caddy**, que permite el acceso externo a las webs.

El resto de las máquinas de la red están aisladas frente accesos no permitidos, solo se puede acceder a ellas desde la red LAN o la VPN, y además solo aceptan conexiones **ssh** con autenticación por clave pública.

Una vez con esto establecido podíamos centrarnos en el desarrollo del proyecto con los principios de IaC.

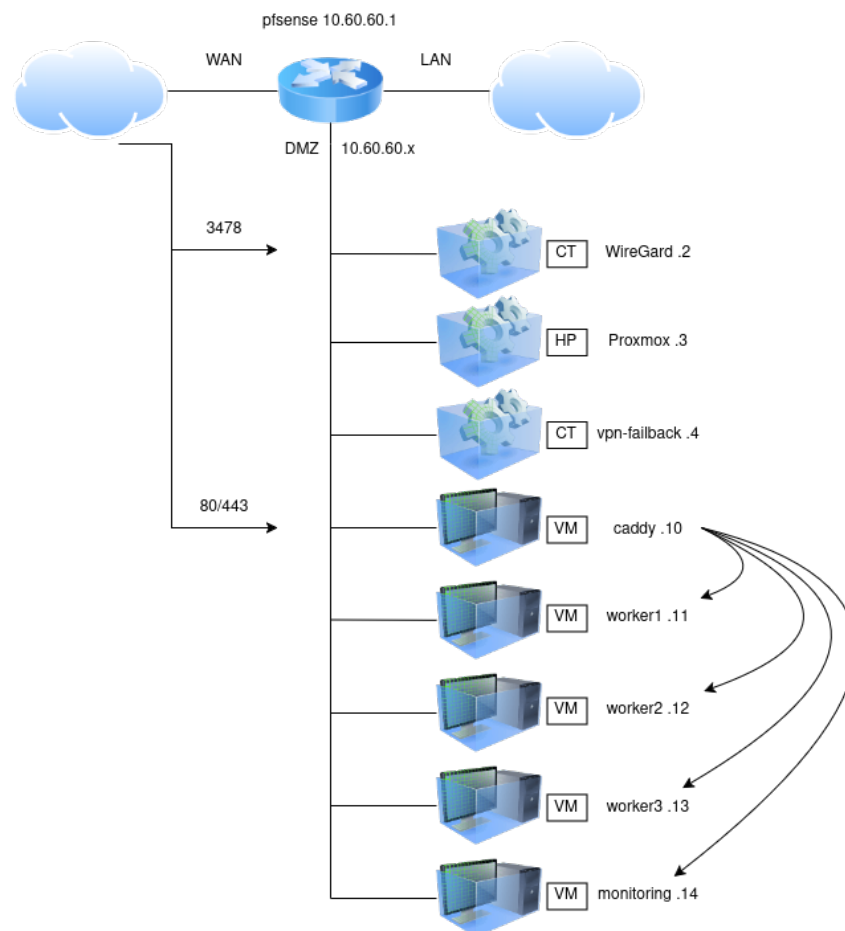


Figura 1: Esquema de red

6. Estructura del proyecto

Organizamos el proyecto en 3 carpetas:

```
.
├── grafana
│   ├── dashboard.json
│   └── load-dashboard.sh
├── nixos
│   ├── deploy-new-host.sh
│   └── first-deploy.sh
```

```

├── flake.lock
├── flake.nix
├── hosts
│   ├── caddy
│   │   ├── default.nix
│   │   ├── disk-config.nix
│   │   └── hardware-configuration.nix
│   ├── monitoring
│   │   └── ...
│   ├── worker1
│   │   └── ...
│   ├── worker2
│   │   └── ...
│   ├── worker3
│   │   └── ...
├── modules
│   ├── caddy-config
│   │   └── Caddyfile
│   ├── caddy.nix
│   ├── common.nix
│   ├── ddns.nix
│   ├── docker.nix
│   ├── fail2ban.nix
│   ├── grafana.nix
│   ├── neovim.nix
│   ├── nginx-indexes
│   │   └── ...
│   ├── nginx_worker.nix
│   ├── node_exporter.nix
│   ├── prometheus.nix
│   ├── ssh.nix
│   ├── users.nix
│   ├── utils.nix
├── secrets
│   ├── cf-token.age
│   ├── grafana-admin-password.age
│   ├── grafana-secret-key.age
│   └── secrets.nix
└── terraform
    ├── main.tf
    ├── tofu.sh
    └── variables.tf

```

- **nixos**: dentro de esta carpeta se guardan todos los archivos de configuración relacionados con el sistema operativo de los equipos.
- **terraform**: aquí mantenemos la declaración de asignación de recursos a cada máquina (número CPU, memoria RAM, disco, red, etc.).
- **grafana**: guardamos un script y el dashboard para cargarlo en la máquina correspondiente.

7. Desarrollo del proyecto

7.1. Configuraciones iniciales

En primer lugar, se llevó a cabo la configuración de un router **pfSense** [5] virtualizado, lo que nos permitió una gestión eficiente de las redes y su monitorización.

A continuación, se configuró un acceso remoto mediante **Tailscale** [6]; sin embargo, esta solución únicamente permite el acceso de uno de los integrantes del equipo a la red. Por ello, se desplegó una segunda VPN basada en **WireGuard** [16], que posibilita el acceso remoto a ambos miembros del equipo de forma simultánea.

Cabe destacar que Tailscale se instaló como solución provisional para disponer de acceso inmediato mientras se investigaba y configuraba correctamente WireGuard, permaneciendo operativo una vez finalizado dicho proceso como backup.

7.2. Declaración de los recursos

Inicialmente, se optó por utilizar **Terraform** [1] para la declaración de recursos; sin embargo, tras investigar y comparar con **OpenTofu** [2], se decidió migrar a esta última debido a su naturaleza completamente open-source, su comunidad activa y su desarrollo continuo. Cabe destacar que ambas herramientas son totalmente compatibles entre sí, ya que OpenTofu es un *fork* directo de Terraform, por lo que comparten la misma sintaxis y ecosistema de proveedores. Esto garantiza además una mejor integración con otras herramientas de código abierto como **NixOS** [3].

Para que OpenTofu pueda interactuar con el hipervisor, es necesario configurar el proveedor de Proxmox en el archivo `main.tf`. En concreto, se utiliza el proveedor **telmate/proxmox** [20], que ofrece soporte para la API de Proxmox y permite gestionar recursos como máquinas virtuales y contenedores LXC de forma declarativa. La autenticación se realiza mediante un **API Token**, evitando así el uso de credenciales de superusuario (`root`).

```
provider "proxmox" {
  pm_api_url      = var.pm_api_url
  pm_api_token_id = var.pm_api_token_id
  pm_api_token_secret = var.pm_api_token_secret
  pm_tls_insecure = var.pm_tls_insecure
}

locals {
  vms = {
    caddy      = { vmid = 110, ip = "10.60.60.10", cores = 6, memory = 2048 }
    worker1    = { vmid = 111, ip = "10.60.60.11", cores = 1, memory = 2048 }
    worker2    = { vmid = 112, ip = "10.60.60.12", cores = 1, memory = 2048 }
    worker3    = { vmid = 113, ip = "10.60.60.13", cores = 1, memory = 2048 }
    monitoring = { vmid = 114, ip = "10.60.60.14", cores = 2, memory = 4096 }
  }
}

resource "proxmox_vm_qemu" "vms" {
  for_each = local.vms

  vmid      = each.value.vmid
  name      = "nixos-${each.key}"
}
```

```

target_node = "pve"
clone       = "debian-12-template"
full_clone  = true
scsihw      = "virtio-scsi-single"
vm_state    = var.vm_state

cpu {
  cores    = each.value.cores
  sockets  = 1
  type     = "host"
}

memory = each.value.memory

disks {
  scsi {
    scsi0 { disk { storage = "local-lvm", size = "20G", iothread = true } }
    scsi1 { cloudinit { storage = "local-lvm" } }
  }
}

network {
  id      = 0
  model   = "virtio"
  bridge  = "vmbr2" # DMZ
}

ipconfig0 = "ip=${each.value.ip}/24,gw=10.60.60.1"
sshkeys    = var.sshkeys
cipassword  = var.cipassword
}

```

Para simplificar el uso de secretos, utilizamos un script `tofu.sh`, que es un simple wrapper que carga las variables de entorno desde un archivo `.env`, y a continuación ejecuta el binario de OpenTofu con los parámetros necesarios pasados al script. De esta forma, no exponemos las credenciales de la API de Proxmox en el código fuente.

7.3. Creación de la plantilla base

Para la plantilla base se optó por una imagen **Debian 12** con soporte para **Cloud-init** [21]. Inicialmente se intentó utilizar una imagen de **Debian 13** instalada manualmente; sin embargo, se encontraron problemas de compatibilidad con Cloud-init relacionados con cambios en la configuración DNS introducidos en esta versión, lo que impedía el correcto aprovisionamiento de las máquinas virtuales. Tras investigar alternativas, se encontró la imagen oficial Cloud-init de Debian 12, que resultó plenamente funcional, por lo que se optó por ella en lugar de continuar con la instalación manual.

La configuración de **Cloud-init**, declarada desde OpenTofu, se encarga de establecer la red, el usuario y las claves SSH en cada máquina virtual durante el primer arranque. Además, se instala el gestor de paquetes **Nix** [22], necesario para el despliegue posterior de NixOS. De este modo, la plantilla queda lista para *nixificarla*: con las claves SSH inyectadas automáticamente y Nix disponible, NixOS puede tomar el control del sistema sin intervención manual.

7.4. Instalación de NixOS con nixos-anywhere

Una vez que OpenTofu ha levantado las máquinas virtuales con Debian, utilizamos **nixos-anywhere** [7] para realizar la transición a NixOS.

A la primera máquina nos tenemos que conectar por SSH y sacar información del hardware para generar el archivo `hardware-configuration.nix` necesario para la instalación de NixOS.

De la siguiente forma:

```
debian@template:~$ sudo $(which nix-shell) -p nixos-install-tools \
--run "nixos-generate-config --no-filesystems --root /mnt"
debian@template:~$ cat /mnt/etc/nixos/hardware-configuration.nix
# Do not modify this file! It was generated by 'nixos-generate-config'
# and may be overwritten by future invocations. Please make changes
# to /etc/nixos/configuration.nix instead.
{ config, lib, pkgs, modulesPath, ... }:

{
  imports =
    [ (modulesPath + "/profiles/qemu-guest.nix")
    ];

  boot.initrd.availableKernelModules = [ "ata_piix" "uhci_hcd" "virtio_pci"
    "virtio_scsi" "sd_mod" "sr_mod" ];
  boot.initrd.kernelModules = [ ];
  boot.kernelModules = [ "kvm-intel" ];
  boot.extraModulePackages = [ ];

  nixpkgs.hostPlatform = lib.mkDefault "x86_64-linux";
}
```

Esta configuración nos sirve para definir el hardware de todas las máquinas virtuales NixOS, dado que todas ellas comparten el mismo hardware virtualizado.

Con el hardware definido, cada máquina se declara mediante **Nix** [22] y se despliega utilizando **nixos-anywhere** [7], que carga una imagen mínima de NixOS en la RAM de la plantilla Debian, mediante SSH, y procede a instalar NixOS en el disco de cada máquina virtual, copiando la configuración declarada.

En este proceso se utiliza **disko** [17] para el particionado declarativo de los discos, lo que nos permite definir la estructura de almacenamiento de forma sencilla y reproducible.

Las claves SSH de los hosts se generan localmente y se envían a cada máquina durante la instalación, garantizando que cada vez que se crean las máquinas tengan siempre las claves SSH de host esperadas, garantizando así la idempotencia y reproducibilidad de la infraestructura desde el primer despliegue. Además, **agenix** [9] utiliza estas claves para descifrar los secretos necesarios para cada host. Si no se generan las claves de host de forma determinista, cada despliegue crearía nuevas claves, lo que invalidaría los secretos cifrados con las claves anteriores y rompería la reproducibilidad.

Posteriormente, cualquier cambio en la configuración de servicios, usuarios y contenedores Docker se gestiona con **Colmena** [8], lo que permite aplicar modificaciones en segundos sin necesidad de reiniciar la máquina ni reinstalar el sistema operativo.

7.5. Gestión de cambios con Colmena

Una vez realizado el despliegue inicial con **nixos-anywhere**, la herramienta encargada de gestionar el ciclo de vida de la configuración es **Colmena** [8].

A diferencia de **nixos-anywhere**, que instala el sistema desde cero, Colmena aplica cambios incrementales sobre máquinas ya en ejecución mediante SSH, sin necesidad de reiniciar ni reinstalar.

7.5.1. Integración con el flake

Colmena lee su configuración directamente del output `colmena` del `flake.nix`. Cada host se define con su dirección IP (`targetHost`), usuario de despliegue (`targetUser`) y su configuración NixOS, que es exactamente la misma que se usa en `nixosConfigurations`.

En el archivo `flake.nix` la sección de Colmena tiene la siguiente forma:

```
colmenaHive = colmena.lib.makeHive self.outputs.colmena;
colmena = {
  meta = {
    nixpkgs = import nixpkgs { inherit system; };
    specialArgs = { inherit disk inputs; };
  };

  caddy = { name, nodes, pkgs, ... }: {
    deployment = {
      targetHost = "10.60.60.10";
      targetUser = builtins.getEnv "USER";

      # You can build on the target machine
      # buildOnTarget = true;
    };
    imports = caddyModules;
  };

  worker1 = { name, nodes, pkgs, ... }: {
    deployment = {
      targetHost = "10.60.60.11";
      targetUser = builtins.getEnv "USER";
    };
    imports = worker1Modules;
  };

  # worker2, worker3 ...

  monitoring = { name, nodes, pkgs, ... }: {
    deployment = {
      targetHost = "10.60.60.14";
      targetUser = builtins.getEnv "USER";
    };
    imports = monitoringModules;
  };
};
```

7.5.2. Flujo de trabajo habitual

El flujo típico al modificar la configuración de uno o varios hosts es el siguiente:

1. Se edita el módulo o archivo de host correspondiente en el repositorio.
2. Se verifica que la configuración compila correctamente sin desplegar:

```
nix run github:zhaofengli/colmena -- build
```

3. Si la build es exitosa, se despliega el cambio. Para aplicarlo a un único host:

```
nix run github:zhaofengli/colmena -- apply --on <host> --impure
```

Para aplicarlo a toda la infraestructura de forma simultánea:

```
nix run github:zhaofengli/colmena -- apply --impure
```

4. Colmena construye la nueva generación del sistema en la máquina local (o en el propio host si se configura despliegue remoto), la transfiere por SSH y ejecuta `nixos-rebuild switch`, activando los cambios en caliente.

7.5.3. Ventajas operativas

El uso de Colmena frente a otros enfoques de despliegue aporta varias ventajas prácticas en nuestro contexto:

- **Despliegue paralelo:** Colmena puede aplicar cambios a múltiples hosts en paralelo, reduciendo el tiempo total de actualización de la infraestructura completa.
- **Setup sencillo:** no requiere instalar ningún agente en los hosts destino; únicamente necesita acceso SSH con el usuario y clave configurados en `deployment`.
- **Consistencia:** aplicar la misma configuración varias veces produce siempre el mismo resultado, sin efectos secundarios acumulativos.

7.5.4. Configuración SSH para Colmena

Para que Colmena pueda conectarse a cada host por SSH es necesario añadir una entrada en `~/.ssh/config` por cada máquina gestionada:

```
Host 10.60.60.10
  IdentityFile ~/.ssh/<clave_ssh_ed25519>
Host 10.60.60.11
  IdentityFile ~/.ssh/<clave_ssh_ed25519>
Host 10.60.60.12
  IdentityFile ~/.ssh/<clave_ssh_ed25519>
Host 10.60.60.13
  IdentityFile ~/.ssh/<clave_ssh_ed25519>
Host 10.60.60.14
  IdentityFile ~/.ssh/<clave_ssh_ed25519>
```

De este modo Colmena selecciona automáticamente la clave correcta sin intervención manual durante el despliegue.

7.6. Configuración de NixOS

7.6.1. Estructura del flake

El punto de entrada de toda la configuración NixOS es el archivo `flake.nix`, que actúa como manifiesto del proyecto: declara las dependencias externas (inputs) y expone las configuraciones de cada host (outputs). Entre los inputs más relevantes se encuentran `nixpkgs` (el repositorio de paquetes de NixOS), `agenix` [9] (gestión de secretos), `disko` [17] (particionado declarativo) y `colmena` [8] (despliegue remoto).

Cada host queda registrado dos veces: en el output de `nixosConfigurations` (usado por **nixos-anywhere** en el primer despliegue) y en el output de `colmena` (usado para actualizaciones posteriores). Esto garantiza que la misma definición de sistema se emplee independientemente del mecanismo de despliegue.

7.6.2. Módulos compartidos

Para evitar la duplicación de configuración entre hosts, extraemos toda la lógica reutilizable en módulos bajo `nixos/modules/`.

Cada host importa el subconjunto de módulos que necesita desde su `default.nix`.

- `common.nix`

Define la base común a todas las máquinas: zona horaria, locale, bootloader (**GRUB** en modo BIOS para compatibilidad con Proxmox), y el conjunto mínimo de paquetes del sistema (`git`, `htop`, `curl`, etc.). También habilita `nix.settings.experimental-features` para soportar flakes.

- `users.nix`

Declara los usuarios del sistema de forma centralizada. Las claves SSH públicas de los administradores se incluyen directamente en la configuración, de modo que cualquier máquina que importe este módulo acepta las mismas identidades desde el primer arranque. Las contraseñas se deshabilitan forzando el acceso exclusivamente por certificado.

- `ssh.nix`

Declara el tipo de las claves SSH y además endurece la configuración del demonio SSH: deshabilita el login con contraseña (`PasswordAuthentication = false`). Combinado con `fail2ban` [18], forman una firme defensa frente a accesos remotos no autorizados en cada máquina.

- `fail2ban.nix`

Habilita el servicio `fail2ban` [18] con una jail para SSH que bloquea automáticamente direcciones IP tras varios intentos de autenticación fallidos. La configuración de este módulo es común a todos los hosts.

- `utils.nix` y `neovim.nix`

Agrupar herramientas de administración del sistema (`tmux`, `ripgrep`, `jq`, `wget`, etc.) y la configuración personalizada de **Neovim** respectivamente, disponibles en todas las máquinas para facilitar el trabajo de los administradores.

7.6.3. Configuración de hosts

Cada directorio bajo `nixos/hosts/<hostname>/` contiene tres archivos:

- `hardware-configuration.nix`: generado a partir de la plantilla Debian; captura los módulos del kernel necesarios para el hardware virtualizado de Proxmox (`virtio_pci`, `virtio_scsi`, etc.).
- `disk-config.nix`: define el esquema de particiones mediante **disko** [17]. Usamos una tabla **GPT** con una partición de arranque BIOS y una partición raíz `ext4`.
- `default.nix`: configuración específica del host; importa los módulos necesarios, asigna el `hostname` y declara los servicios particulares de esa máquina.

8. Servicios desplegados

8.1. Caddy (proxy inverso)

La máquina `caddy` actúa como punto de entrada a toda la infraestructura. El módulo `caddy.nix` habilita el servicio `services.caddy` de NixOS apuntando a un `Caddyfile` personalizado ubicado en `modules/caddy-config/Caddyfile`. **Caddy** [12] gestiona automáticamente los certificados TLS mediante **ACME/Let's Encrypt** [23], usando el token de **Cloudflare** [24] (almacenado como secreto `age` [10]) para el challenge **DNS-01**, lo que permite obtener certificados incluso sin exponer el puerto 80 directamente.

El `Caddyfile` define los virtual hosts necesarios para enrutar el tráfico entrante hacia los servicios internos de la red DMZ mediante proxy inverso, añadiendo cabeceras de seguridad y terminación TLS en este punto.

Además, hace de balanceador de carga entre los 3 workers, distribuyendo las peticiones entrantes de forma round-robin y permitiendo escalar horizontalmente los servicios desplegados en los workers.

8.2. DDNS (ddns.nix)

Dado que la IP pública del router puede cambiar, el módulo `ddns.nix` configura un servicio de **DNS dinámico** mediante la API de **Cloudflare** [24]. Se despliega como un contenedor de **Docker** (`favonia/cloudflare-ddns:latest`) que comprueba la IP pública actual y actualiza el registro DNS correspondiente en caso de cambio, usando de nuevo el token de Cloudflare gestionado por `agenix` [9].

8.3. Workers con Nginx (nginx_worker.nix)

Los hosts `worker1`, `worker2` y `worker3` son máquinas de propósito general que ejecutan servicios sobre Docker [11]. El módulo `nginx_worker.nix` levanta un servidor **Nginx** [13] en cada worker que sirve una página de índice personalizada (los archivos `index1.html`, `index2.html` e `index3.html` ubicados en `modules/nginx-indexes/`), permitiendo identificar visualmente cada nodo a través del proxy inverso Caddy.

8.4. Docker (docker.nix)

El módulo `docker.nix` habilita el demonio **Docker** [11] en los hosts que lo requieren mediante `virtualisation.docker.enable = true`, configura el usuario `docker` con los permisos nece-

sarios y define los contenedores que deben arrancarse automáticamente en el boot del sistema mediante unidades `systemd` que invocan `docker compose up`. Los archivos `compose.yml` de cada servicio se despliegan en `/srv/docker/<servicio>/` y los secretos necesarios se montan desde `/run/agenix/` en modo solo lectura.

8.5. Monitorización: Prometheus, Node Exporter y Grafana

La máquina `monitoring` centraliza la observabilidad de toda la infraestructura mediante tres módulos:

- `node_exporter.nix`: habilitado en *todos* los hosts, expone métricas del sistema (CPU, memoria, disco, red) en el puerto 9100 mediante `services.prometheus.exporters.node` [14].
- `prometheus.nix`: exclusivo de `monitoring`. Configura `services.prometheus` [14] con un `scrapeConfig` apuntando a los node exporters de cada worker, el node exporter del host Caddy, el del router y las métricas del Caddy. Intervalo de scraping y retención declarados en este módulo.
- `grafana.nix`: también exclusivo de `monitoring`, levanta `services.grafana` [15] con Prometheus como DataSource provisionado automáticamente. Los dashboards se configuran a posteriori mediante una llamada a la API de Grafana y se exponen al exterior a través del proxy inverso Caddy.

Esta arquitectura permite visualizar en tiempo real el estado de toda la granja de servidores desde un único panel sin necesidad de configuración manual una vez desplegada la infraestructura.

9. Ventajas y limitaciones

9.1. Ventajas

- **Reproducibilidad total:** Podemos recrear toda la infraestructura desde cero en cuestión de minutos usando únicamente los archivos del repositorio `git`.
- **Seguridad:** Gracias a `agenix` [9], los secretos (tokens de Cloudflare, contraseñas de Docker) viajan cifrados en el repositorio y solo se descifran en el tiempo de ejecución en la memoria RAM de la VM.
- **Gestión unificada:** La combinación de NixOS [3] y Colmena [8] permite gestionar una granja de servidores como si fuera una sola entidad.

9.2. Limitaciones y desafíos

- **Curva de aprendizaje:** El ecosistema Nix (Flakes, lenguaje Nix) tiene una curva de aprendizaje pronunciada comparado con métodos tradicionales de administración.
- **Calidad del Provider:** El proveedor de Proxmox [4] para OpenTofu [2]/Terraform [1] presenta limitaciones en la gestión de estados complejos. En ocasiones, cambios menores en la configuración de hardware provocan largas esperas para que las VMs vuelvan a estar disponibles, esto es acentuado por el hardware que empleamos (los discos duros y controladora principalmente).
- **Automatización incompleta:** No hemos logrado una “automatización de un solo clic” (*full automation*); todavía se requiere la creación manual de la API Key en la interfaz de Proxmox y la captura manual del `hardware-configuration.nix` antes del primer

despliegue, además por incompatibilidades de Grafana utilizamos su API rest para la configuración del dashboard, lo cual viola el patrón declarativo.

10. Conclusiones

La realización de este trabajo ha permitido demostrar que es posible construir una infraestructura completamente declarativa y reproducible sobre hardware commodity, combinando herramientas del ecosistema open-source sin depender de soluciones propietarias.

10.1. Pipeline de despliegue

El resultado más tangible del proyecto es un pipeline de despliegue completo y reproducible que parte de un repositorio `git` vacío y culmina en una infraestructura completamente operativa:

1. **Provisión de recursos (OpenTofu [2]):** a partir de `main.tf`, OpenTofu declara y crea las cinco máquinas virtuales en Proxmox [4] con sus recursos asignados (CPU, RAM, disco, red), clonando la plantilla Debian 12 con Cloud-init [21].
2. **Bootstrap del sistema (Cloud-init):** en el primer arranque, Cloud-init configura la red, inyecta las claves SSH y prepara el entorno Nix [22] necesario para el siguiente paso.
3. **Instalación de NixOS (nixos-anywhere [7] + disko [17]):** el script `first-deploy.sh` orquesta la instalación remota de NixOS en cada máquina mediante SSH. Durante este paso se transfieren las claves de host precalculadas, garantizando que **agenix** [9] pueda descifrar los secretos desde el primer arranque.
4. **Gestión de cambios (Colmena [8]):** una vez instalado NixOS, cualquier modificación en la configuración se aplica de forma incremental sobre las máquinas en ejecución mediante `colmena apply`, sin necesidad de reinstalar ni reiniciar los hosts.

10.2. Valoración global

El enfoque adoptado demuestra que IaC declarativo no se limita a la provisión de recursos: extendido hasta el nivel del sistema operativo mediante NixOS [3], permite gestionar una granja de servidores como si fuera un único artefacto versionado. La combinación de OpenTofu [2], `nixos-anywhere` [7] y Colmena [8] cubre el ciclo de vida completo de la infraestructura con coherencia declarativa en cada fase.

Los principales objetivos propuestos se han cumplido: la infraestructura es reproducible, los secretos viajan cifrados en el repositorio, el despliegue es automático a partir del código y la configuración de todos los servicios está centralizada y versionada.

Como trabajo futuro, los pasos naturales serían eliminar los dos pasos manuales restantes: la creación de la API Key en Proxmox y la provisión del dashboard de Grafana [15], para alcanzar una automatización de extremo a extremo completamente declarativa.

Referencias

- [1] HashiCorp. *Terraform*. Accedido: 2026. URL: <https://www.terraform.io>.
- [2] OpenTofu contributors. *OpenTofu*. Accedido: 2026. URL: <https://opentofu.org>.
- [3] NixOS contributors. *NixOS*. Accedido: 2026. URL: <https://nixos.org>.
- [4] Proxmox Server Solutions GmbH. *Proxmox Virtual Environment*. Accedido: 2026. URL: <https://www.proxmox.com>.
- [5] Netgate. *pfSense — Open Source Firewall & Router*. <https://www.pfsense.org>. Accedido: 2026.
- [6] Tailscale Inc. *Tailscale — Zero config VPN*. <https://tailscale.com>. Accedido: 2026.
- [7] nix-community. *nixos-anywhere*. Accedido: 2026. URL: <https://github.com/nix-community/nixos-anywhere>.
- [8] zhaofengli. *Colmena*. Accedido: 2026. URL: <https://github.com/zhaofengli/colmena>.
- [9] ryantm. *agenix*. Accedido: 2026. URL: <https://github.com/ryantm/agenix>.
- [10] Filippo Valsorda. *age*. Accedido: 2026. URL: <https://github.com/FiloSottile/age>.
- [11] Docker, Inc. *Docker*. Accedido: 2026. URL: <https://www.docker.com>.
- [12] Caddy Web Server. *Caddy — The Ultimate Server*. <https://caddyserver.com>. Accedido: 2026.
- [13] F5, Inc. *nginx — HTTP and reverse proxy server*. <https://nginx.org>. Accedido: 2026.
- [14] Prometheus Authors. *Prometheus — Monitoring system & time series database*. <https://prometheus.io>. Accedido: 2026.
- [15] Grafana Labs. *Grafana — The open observability platform*. <https://grafana.com>. Accedido: 2026.
- [16] Jason A. Donenfeld. *WireGuard: fast, modern, secure VPN tunnel*. <https://www.wireguard.com>. Accedido: 2026.
- [17] nix-community. *disko — Declarative disk partitioning and formatting using Nix*. <https://github.com/nix-community/disko>. Accedido: 2026.
- [18] Fail2ban Project. *Fail2ban — Intrusion prevention software*. <https://www.fail2ban.org>. Accedido: 2026.
- [19] Proxmox Server Solutions GmbH. *Proxmox VE — Linux Containers (LXC)*. https://pve.proxmox.com/wiki/Linux_Container. 2026.
- [20] Telmate. *Terraform Provider for Proxmox*. <https://github.com/Telmate/terraform-provider-proxmox>. Accessed: 2026.
- [21] Cloud init Contributors. *Cloud-init: The standard for customising cloud instances*. <https://cloud-init.io/>. Accessed: 2026.
- [22] Nix contributors. *Nix*. Accedido: 2026. URL: <https://nixos.org>.
- [23] Internet Security Research Group (ISRG). *Let's Encrypt — Free, automated, and open certificate authority*. <https://letsencrypt.org>. Accedido: 2026.
- [24] Cloudflare, Inc. *Cloudflare — The Web Performance & Security Company*. <https://www.cloudflare.com>. Accedido: 2026.